1 Py-Tracker Core Pipeline

Py-Tracker Core is a streamlined version of the original Py-Tracker branch, with simplified components, revised semantics, and additional analysis and automation modules.

1.1 Mutant Generation

Mutants are generated for an already checked-out repository using the CodeMutate tool. After installing CodeMutate, the command-line interface provided by Py-Tracker Core replaces the original CodeMutate interface in order to ensure compatibility with the downstream tracking pipeline. First-order mutants for a given module are produced by invoking:

```
python cmutate -t ../repo/src/mymodule.py -o ../data/mu_mymodule.json
```

This process yields a JSON file containing the set of generated mutants for the specified source file.

1.2 Commit Selection

The commits to be analyzed are selected using a dependency-aware history filtering procedure rather than a simple chronological or filename-based scan. Given one or more target source files, the selection process traverses the repository history along the first-parent chain starting from `HEAD` and retains only commits that are relevant to the evolution of those targets.

For each examined commit, the repository state at that revision is reconstructed and a lightweight Python import graph is derived from the tracked `.py` files. Based on this graph, a relevance neighborhood is computed for every target file. This neighborhood contains the target itself, the files it directly imports, the files that directly import it, and test files that import any of these related modules. A commit is considered relevant if it modifies at least one file in this neighborhood.

Independently of the relevance criterion, the current `HEAD` commit is always included in the resulting sequence. Additionally, when a boundary commit is specified (e.g., via an explicit starting point), that boundary commit is also included even if it does not satisfy the relevance condition. This ensures that both the latest project state and the initial reference point of the analysis are always preserved.

This strategy yields a focused commit sequence centered on the target module and its immediately related implementation and test context, thereby avoiding the inclusion of unrelated historical commits from the same repository.

The selection procedure may additionally be bounded either by a maximum number of retained commits, by an explicit boundary commit, or by a time-based window. The traversal always follows the mainline history using the first-parent chain, which produces a linear sequence of commits suitable for downstream mutant tracking and test execution.

An example usage is shown below:

```
python select_relevant_commits.py \
  --repo ../../chk/youtube-dl-41b/youtube-dl \
  --targets youtube_dl/utils.py \
  --max-commits 25 \
  --output commits_to_run.txt \
  --evolution-output commit_evolution.json
```

The resulting `commits_to_run.txt` file contains the selected commits in reverse chronological order, with the first entry corresponding to `HEAD`. In addition, the optional `commit_evolution.json` file records, for each retained commit, a summary of source and test evolution, including added, modified, deleted, and renamed Python files.

1.3 Mutant Tracking

Given the selected commit sequence and the generated mutant files, mutant propagation across historical versions is computed using `mutant_backward_tracker.py`, together with the module `gumtree_mutant_mapper.py`, which provides GumTree-based abstract syntax tree differencing and node mapping functionality.

The tracker traverses the explicitly provided commit list and maps mutant locations between successive file versions. For each commit transition and each modified file, the AST mapping is computed once and reused across all mutants in that file, thereby substantially reducing computational overhead.

An example invocation is shown below:

```
python mutant_backward_tracker.py \
  --repo ../../chk/networkx-4b/thefuck \
  --commits-file ../../chk/networkx/data/commits_to_run.txt \
  --mutant-json ../../chk/networkx/data/mutants/*.json \
  --output ../../chk/networkx/data/mutant_history.json
```

The output of this step is a structured history file that records, for each mutant, its propagated information across commits.

1.4 Test Execution

Using the computed mutant histories, tests are executed across the selected commits via a test runner built on top of the `pytest-simple-ndjson-report` plugin. This plugin is a simplified derivative of `pytest-mutation-runner`. In contrast to the original mutation runner, it omits features such as early termination upon mutant kill and instead focuses on reliably recording detailed per-test and per-mutant execution outcomes in a structured NDJSON format. An example invocation is shown below:

```
python commit_test_runner.py \
  --repo "$REPO" \
  --commit "$COMMIT" \
  --conda-env "$ENV_NAME" \
  --pkg-name "$PKG_NAME" \
  --results "$OUTDIR" \
  --run "both" \
  --mutant-history ../../chk/networkx/data/mutant_history.json
```

Additional execution scripts are provided in the `scripts/` directory, including configurations for high-performance computing environments and options for specifying ignored tests, explicit test subsets, and execution timeouts.

1.5 Result Post-processing

The execution traces collected during test runs are post-processed to obtain mutation testing outcomes. This step labels mutants as killed, survived, or inactive and reshapes the data into formats suitable for analysis.

Post-processing is performed using the script `mutant_results_postprocess.py`, which applies configurable filtering and aggregation procedures. The resulting data can then be used to compute mutation scores, survival statistics, and other summary measures for the empirical study.

1.5.1 Mutant Status Aggregation

Mutation outcomes are derived by aggregating per-test execution results into a single status per mutant. Each mutant is associated with a set of test outcomes, where each test is labeled with a status such as `killed`, `survived`, `incompetent`, `anomalous`, or `unknown`.

The aggregation procedure applies a priority-based rule over the set of test statuses. The current implementation follows the precedence:

$$\texttt{incompetent} > \texttt{killed} > \texttt{anomalous} > \texttt{survived} > \texttt{unknown}$$

That is, a mutant is classified as:

- `incompetent` if any test execution is marked as incompetent,
- `killed` if no incompetent outcome exists but at least one test kills the mutant,
- `anomalous` if no kill is observed but anomalous behavior is detected,
- `survived` if all observed outcomes are survived or unknown, and at least one test explicitly survives,
- `unknown` otherwise.

This aggregation is implemented in the function `mutant_status_from_tests`, which operates over the per-test result dictionary.

Configurability and Limitations. The aggregation logic is intentionally simple and may not fully capture all semantic nuances of mutation outcomes. In particular, the interpretation of `anomalous` and `incompetent` statuses, as well as their relative priority, may require refinement depending on the experimental setting.

The current implementation also includes alternative formulations (e.g., stricter definitions of `survived`) that can be enabled or disabled by modifying the aggregation logic. As a result, users should be aware that mutation status classification is not fixed and may evolve as the framework is extended or adapted.

Usage in Analysis Tools. The aggregation logic described above is used across multiple analysis and visualization utilities within the framework. In particular, scripts such as `mutant_matrix_inspector.py`, `mutant_kill_explorer.py`, and `viz_mutants_over_time.py` rely on this classification to interpret and present mutation outcomes.

These tools use the aggregated mutant status to construct visualizations (e.g., mutation matrices over time), inspect individual mutant behavior, and analyze test effectiveness. As a result, any modification to the aggregation policy may directly affect the interpretation and visualization of results produced by these components.

Project-Specific Execution Although the core pipeline aims to provide a uniform workflow across projects, some repositories require project-specific execution and normalization logic. This is particularly the case for repositories whose test layout or pytest invocation pattern differs from the more common single-file style execution.

For many projects, tests can be executed using direct pytest targets such as:

```
pytest hello_test.py --ignore bye_test.py
```

However, some projects, such as `tqdm`, require glob-style test selection rooted in a dedicated test directory, for example:

```
pytest tests/test_*.py --ignore something_test.py
```

In such cases, the collected test identifiers may differ between program-mode and mutant-mode execution. In particular, mutant-mode execution may introduce additional prefixes or other path-related changes in the reported test IDs, whereas program-mode execution retains the normal pytest identifiers. Since downstream analysis relies on matching corresponding tests across these two execution modes, additional normalization logic is required.

To support such cases, the framework includes project-specific variants such as `commit_test_runner_tqdm.py`, `mutant_results_postprocess_for_tqdm.py`, and helper scripts such as `tqdm_1b_wo_pck.sh`. These components adapt the execution and post-processing steps to the repository-specific behavior of `tqdm`, including customized test selection and alignment of test identifiers across execution modes.

These variants should be understood as compatibility layers rather than as part of the default pipeline. For new projects, the standard `commit_test_runner.py` and `mutant_results_postprocess.py` workflow should be preferred unless repository-specific test collection behavior makes additional handling necessary.

Repository Organization. Project-specific execution and post-processing variants are located under the `project_specific/` directory in the codebase.

1.6 Interactive Review and Event Curation

After mutation results have been post-processed, we perform a manual review step to identify mutants whose status transitions appear informative. This review is supported by two complementary interactive tools.

The first-stage reviewer, `review_interesting_events.py`, is used to scan candidate mutants and identify potentially interesting cases. The tool builds, for each mutant, a status timeline over the selected commits using the aggregated mutation outcomes produced during post-processing. It highlights transition points of interest, especially changes from `survived` to `killed`/`incompetent` and from `killed`/`incompetent` to `survived`. During inspection, the reviewer can examine commit-level mutant outcomes, failing program tests, commit-evolution metadata, the bug patch, and a reconstructed unified diff of the mutant at a selected commit. Mutants judged to be potentially informative are then saved together with preliminary labels and optional notes.

The second-stage reviewer, `saved_mutant_reviewer.py`, is used to revisit previously saved mutants and record structured event details for the selected labels. Unlike the first-stage reviewer, which is primarily intended for discovery and coarse filtering, this second tool operates on an explicit input list of already selected mutants. For each mutant, it guides the reviewer through the capture of specific commits associated with the labeled transition pattern. For example, depending on the assigned label, the reviewer may record the last commit in which the mutant survived, the first commit in which it was killed, or the corresponding boundary commits for more complex patterns such as `survived` $\rightarrow$ `killed` $\rightarrow$ `survived`. These event details are then stored together with the saved mutant record.

This two-stage review process separates discovery from structured annotation. The first pass is used to find mutants worth further attention, while the second pass is used to curate precise transition events for downstream analysis. The resulting saved records contain the mutant identifier, manually assigned labels, optional reviewer notes, and, when available, structured event details that localize the relevant status transitions in the commit history.

1.6.1 Alternative Inspection Tools.

While the two-stage reviewer workflow is the primary mechanism for identifying and annotating interesting mutants, alternative visualization tools are also available. In particular, `mutant_matrix_inspector.py` provides a more direct

view of the mutation matrix and supports fully manual inspection and selection. Although this approach requires additional effort, it can be useful when the automated candidate selection or interactive reviewers are not sufficient. These tools are therefore complementary, and the choice between them depends on the analysis scenario.

1.7 Saved-Mutant Review and LLM-Assisted Interpretation

After interesting mutants and their transition events have been curated, we perform a deeper review stage using two complementary tools: `mutant_review_gui.py` and `mutant_review_cli.py`. These tools operate on the saved-mutant records produced by the earlier review stages and are intended to support detailed inspection and downstream interpretation of status transitions.

The graphical reviewer, `mutant_review_gui.py`, provides an interactive environment for browsing saved mutants and their associated transition events. For each selected mutant, the tool displays the mutant metadata, the recorded event structure, reconstructed mutant source at the relevant commits, changed test outcomes across event boundaries, and side-by-side diffs for Python files modified between the commits associated with the event. This interface is used for additional manual inspection of previously saved cases and for validating that the recorded event boundaries and supporting evidence are coherent.

The command-line reviewer, `mutant_review_cli.py`, exports the same underlying evidence in a structured textual and JSON form suitable for further analysis. For a selected mutant and event, it retrieves the associated commits, their commit messages, reconstructed mutant spans, commit-level test results, test-result differences across event boundaries, and source-code diffs for the changed Python files. Depending on the selected options, it can also emit a localized code window around the mutant, a unified diff between reconstructed mutant versions, or the full reconstructed mutant source. This representation is used both for manual inspection and for preparing inputs to downstream LLM-based analysis.

Using the exported evidence, we then ask a large language model (LLM) to explain the most likely reason for a mutant's status transition across commits. The LLM is instructed to use only the supplied evidence and to distinguish among causes such as changed reachability or observability, changed test oracle, changed surrounding semantics, and unrelated or noisy effects. The prompt also explicitly states that a status transition may be caused by changes outside the mutated line and that, when reconstructed mutant code remains unchanged across commits, explanations based on observability or oracle changes should be preferred over unsupported semantic claims.

More specifically, the LLM is instructed to:

- identify the most likely reason for the status transition,
- classify the cause into one of the predefined explanatory categories,
- ground every claim in the provided evidence,

- avoid unsupported speculation, and
- return the result in a predefined JSON schema.

The purpose of this stage is not to replace human judgment, but to support it. The generated explanations are reviewed and validated manually, and the final categorization is based on this combined human-in-the-loop process. As a result, the workflow includes both a curation phase, in which interesting mutants and events are selected, and an interpretation phase, in which those curated cases are examined in greater depth and assigned a reasoned explanation for their status changes.

1.7.1 Cross-Bug Deduplication Analysis

As an optional downstream analysis step, we aggregate mutants across all bug instances within the same project and deduplicate recurring cases. For each deduplicated mutant, we retain the set of bug instances in which it appears, thereby producing a project-level view in addition to the original bug-level results.

This deduplicated analysis is mainly used for oracle-related, semantic, and mixed categories, where the same underlying mutant may recur meaningfully across instances. By contrast, other cases e.g., observability-related are kept in their raw, per-instance form.

Accordingly, we maintain both a raw analysis view and a deduplicated project-level view. The former preserves the original bug-instance context, whereas the latter highlights recurring mutant patterns across the project.

1.8 Commit Selection (Legacy)

The commits to be analyzed are extracted using the script `list_commits_since.py`. This script traverses the mainline history of the repository using the first-parent chain, thereby producing a linear sequence of commits from `HEAD` backwards to a user-defined boundary.

The boundary can be specified in one of two mutually exclusive ways. First, a time-based window may be provided using the `-days` option, which selects commits from `HEAD` back to the closest mainline commit at or before `HEAD` minus the given number of days. Alternatively, a specific commit may be supplied via `-start`. If this commit lies on the mainline history, the traversal stops exactly at that commit. Otherwise, the closest newer mainline commit is selected based on commit timestamps.

Commit selection further supports filtering based on either exact filename matches or glob-style pattern matching. Example invocations include:

```
python list_commits_since.py ../repo/ \
  --start f372f3d56cecb3b00ccee6f3b74935aece4ba013 \
  --patterns "test_*.py" \
  > ../data/commits_to_run.txt
```

```
python list_commits_since.py ../repo/ \
  --days 365 \
  --files fish.py \
  > ../data/commits_to_run.txt
```

The resulting file `commits_to_run.txt` lists commits in reverse chronological order, with the first entry corresponding to `HEAD`.